Restaurant Analytics Using Hadoop and PySpark

1. Setup & Data Upload in hadoop

Step 1: Start Hadoop

start-dfs.sh
start-yarn.sh

Step 2: Create HDFS Directory and Upload File

hadoop fs -put ‘/home/cloudera/Desktop/restaurant.csv’ /user/cloudera/Sivanandhini/

Step 3: Verify Upload

hadoop fs -ls /user/cloudera/Sivanandhini/

Step 4: Create SparkSession and Load Data in Pyspark

Step 5: Case Studies:

1. Global Cuisine Popularity Analysis

```
>>> df_cuisine = df.withColumn("Cuisine", explode(split(col("Cuisines"), ", ")))
>>>
>>>
>>> global_cuisine = df_cuisine.groupBy("Cuisine").count().orderBy(desc("count"))
>>>
>>>
>>> city_cuisine = df_cuisine.groupBy("City", "Cuisine").count() \
...                           .orderBy("City", desc("count"))
```

```
>>> global_cuisine.show(3)
+------------+-----+
|     Cuisine|count|
+------------+-----+
|North Indian| 3958|
|     Chinese| 2733|
|   Fast Food| 1984|
+------------+-----+
only showing top 3 rows

>>> city_cuisine.show(5)
+---------+------------+-----+
|     City|     Cuisine|count|
+---------+------------+-----+
|Abu Dhabi|      Indian|    7|
|Abu Dhabi|    American|    4|
|Abu Dhabi|     Chinese|    3|
|Abu Dhabi|North Indian|    3|
|Abu Dhabi|    Japanese|    2|
```

2. Normalizing and Comparing Costs Across Currencies

Data Cleaning, Fixing Encoding issues and adding currency exchange rates:

```
>>> from pyspark.sql.functions import col
>>>
>>> df = df.filter(~col("Currency").isin("Yes", "No"))
>>> from pyspark.sql.functions import when
>>>
>>> df = df.withColumn(
...     "Currency",
...     when(col("Currency").like("%Pounds%"), "Pounds(£)")
...     .when(col("Currency").like("%Rupiah%"), "Indonesian Rupiah(IDR)")
...     .otherwise(col("Currency"))
... )
>>> exchange_rates = {
...     "Indian Rupees(Rs.)": 0.012,
...     "Dollar($)": 1.0,
...     "Pounds(£)": 1.25,
...     "Qatari Rial(QR)": 0.27,
...     "Sri Lankan Rupee(LKR)": 0.0031,
...     "NewZealand($)": 0.62,
...     "Emirati Diram(AED)": 0.27,
...     "Turkish Lira(TL)": 0.031,
...     "Rand(R)": 0.053,
...     "Indonesian Rupiah(IDR)": 0.000064,
...     "Botswana Pula(P)": 0.074,
```

Converting Cost to USD:

```
>>> df = df.withColumn(
...     "Cost_USD",
...     col("avg_cost_for_two") * mapping_expr[col("Currency")]
... )
>>> from pyspark.sql.functions import create_map, lit
>>> from itertools import chain
>>>
>>> mapping_expr = create_map([lit(x) for x in chain(*exchange_rates.items())])
>>>
>>> df = df.withColumn(
...     "Cost_USD",
...     col("avg_cost_for_two") * mapping_expr[col("Currency")]
... )
>>> df = df.filter(col("Cost_USD").isNotNull())
>>> from pyspark.sql.functions import avg, round
>>>
>>> df.groupBy("City") \
...   .agg(round(avg("Cost_USD"), 2).alias("avg_cost_usd")) \
...   .orderBy(col("avg_cost_usd").desc()) \
...   .show(5)
```

Most Expensive cities:

```
>>> df.groupBy("City") \
...   .agg(round(avg("Cost_USD"), 2).alias("avg_cost_usd")) \
...   .orderBy(col("avg_cost_usd").desc()) \
...   .show(5)
+----------------+------------+
|            City|avg_cost_usd|
+----------------+------------+
|      Pasay City|      246.67|
|       Singapore|      158.95|
|Mandaluyong City|       148.0|
|     Taguig City|      127.65|
|     Paynesville|       120.0|
+----------------+------------+
only showing top 5 rows
```

Most Affordable Cities:

```
>>> df.groupBy("City") \
...   .agg(round(avg("Cost_USD"), 2).alias("avg_cost_usd")) \
...   .orderBy(col("avg_cost_usd").asc()) \
...   .show(5)
+---------+------------+
|     City|avg_cost_usd|
+---------+------------+
|??stanbul|        2.52|
|   Ankara|         2.7|
|Faridabad|        5.37|
| Amritsar|        5.77|
| Varanasi|        6.06|
+---------+------------+
only showing top 5 rows
```

Most Expensive Country

```
>>> df.groupBy("Country Code") \
...    .agg(round(avg("Cost_USD"), 2).alias("avg_cost_usd"))
...    .orderBy(col("avg_cost_usd").desc()) \
...    .show()
+------------+------------+
|Country Code|avg_cost_usd|
+------------+------------+
|         184|      158.95|
|         162|       118.9|
|         166|       60.41|
|         215|       59.77|
|         214|       45.47|
|         148|       43.24|
|          37|       36.25|
|          30|       26.93|
|         216|       26.35|
|          14|       24.08|
|         189|       22.25|
|          94|        18.0|
|           1|        7.48|
|         191|        7.36|
|         208|        2.63|
+------------+------------+
```

mouse pointer inside or press Ctrl+G.

rch

Most Affordable Country:

```
>>> df.groupBy("Country Code") \
...    .agg(round(avg("Cost_USD"), 2).alias("avg_cost_usd")) \
...    .orderBy(col("avg_cost_usd").asc()) \
...    .show(10)
+------------+------------+
|Country Code|avg_cost_usd|
+------------+------------+
|         208|        2.63|
|         191|        7.36|
|           1|        7.48|
|          94|        18.0|
|         189|       22.25|
|          14|       24.08|
|         216|       26.35|
|          30|       26.93|
|          37|       36.25|
|         148|       43.24|
+------------+------------+
only showing top 10 rows
```

3. Impact of Services on Customer Ratings

Grouping only Online Delivery:

```
>>> from pyspark.sql.functions import avg, round
>>>
>>> df.groupBy("Has Online delivery") \
...   .agg(round(avg("rating"), 2).alias("avg_rating")) \
...   .show()
+-------------------+----------+
|Has Online delivery|avg_rating|
+-------------------+----------+
|                 No|      2.46|
|                Yes|      3.25|
+-------------------+----------+
```

Analysing Table booking only:

```
>>> df.groupBy("Has Table booking") \
...   .agg(round(avg("rating"), 2).alias("avg_rating")) \
...   .show()
+-----------------+----------+
|Has Table booking|avg_rating|
+-----------------+----------+
|               No|      2.56|
|              Yes|      3.44|
+-----------------+----------+
```

Analysing Both :

```
>>> df.groupBy("Has Online delivery", "Has Table booking") \
...   .agg(round(avg("rating"), 2).alias("avg_rating")) \
...   .orderBy(col("avg_rating").desc()) \
...   .show()
+-------------------+-----------------+----------+
|Has Online delivery|Has Table booking|avg_rating|
+-------------------+-----------------+----------+
|                Yes|              Yes|       3.6|
|                 No|              Yes|      3.35|
|                Yes|               No|      3.17|
|                 No|               No|      2.36|
+-------------------+-----------------+----------+
```

Analysing Weighted Rate of both:

```
>>> df.groupBy("Has Online delivery", "Has Table booking") \
...   .agg(
...       round(
...           sum(col("rating") * col("Votes")) / sum(col("Votes")),
...           2
...       ).alias("weighted_rating")
...   ).orderBy(col("weighted_rating").desc()) \
...   .show()
+-------------------+-----------------+---------------+
|Has Online delivery|Has Table booking|weighted_rating|
+-------------------+-----------------+---------------+
|                 No|               No|           4.08|
|                Yes|              Yes|           3.95|
|                 No|              Yes|           3.91|
|                Yes|               No|           3.75|
+-------------------+-----------------+---------------+
```

4. Identifying "Hidden Gems" (Weighted Rating Analysis)

```
>>> rated_res = df.select("Aggregate rating", "Restaurant Name","Votes").filter(
(col("Aggregate rating") >= 4) & (col("Votes") >= 1000)).orderBy("Votes",ascendi
ng= False).show()
+----------------+--------------------+-----+
|Aggregate rating|     Restaurant Name|Votes|
+----------------+--------------------+-----+
|             4.8|                Toit|10934|
|             4.8|                Toit|10934|
|             4.8|                Toit|10934|
|             4.7|            Truffles| 9667|
|             4.7|            Truffles| 9667|
|             4.7|            Truffles| 9667|
|             4.3|    Hauz Khas Social| 7931|
|             4.3|    Hauz Khas Social| 7931|
|             4.3|    Hauz Khas Social| 7931|
|             4.3|    Hauz Khas Social| 7931|
|             4.3|           Peter Cat| 7574|
```

5. Restaurant Clustering for Marketing (PySpark MLlib)

Command:

```
>>> from pyspark.sql.functions import when, col
>>>
>>> df_clustered = df.withColumn(
...     "Segment",
...     when((col("Cost_USD") > 50) & (col("Aggregate rating") >= 4), "Luxury Di
ning")
...     .when((col("Cost_USD") > 20), "Mid-range")
...     .otherwise("Budget Eats")
... )
>>>
>>> df_clustered.select("Restaurant Name", "Segment").show(10)
```

Output:

```
>>> df_clustered.select("Restaurant Name", "Segment").show(10)
+--------------------+-------------+
|     Restaurant Name|      Segment|
+--------------------+-------------+
|    Le Petit Souffle|Luxury Dining|
|    Le Petit Souffle|Luxury Dining|
|    Le Petit Souffle|Luxury Dining|
|    Izakaya Kikufuji|Luxury Dining|
|Heat - Edsa Shang...|Luxury Dining|
|Heat - Edsa Shang...|Luxury Dining|
|Heat - Edsa Shang...|Luxury Dining|
|Heat - Edsa Shang...|Luxury Dining|
|                Ooma|Luxury Dining|
|                Ooma|Luxury Dining|
+--------------------+-------------+
only showing top 10 rows
```

6. Geolocation-based Food Hub Detection

Rounding the Latitude and Longitude for grouping the restaurant into a hub:

Output:

```
+-------+-------+-----+
|lat_bin|lon_bin|count|
+-------+-------+-----+
|    0.0|    0.0|  969|
|  28.63|  77.22|  388|
|  28.57|  77.32|  273|
|  28.65|  77.12|  213|
|  28.59|  77.17|  200|
|  28.49|  77.09|  158|
|  28.57|  77.33|  155|
|  28.53|  77.22|  145|
|  28.64|  77.18|  140|
|   28.6|  77.23|  132|
+-------+-------+-----+
only showing top 10 rows
```

Getting the Count with City name:

```
>>> df_geo.groupBy("City", "lat_bin", "lon_bin") \
...       .count() \
...       .orderBy(col("count").desc()) \
...       .show(10)
+---------+-------+-------+-----+
|     City|lat_bin|lon_bin|count|
+---------+-------+-------+-----+
|New Delhi|    0.0|    0.0|  436|
|New Delhi|  28.63|  77.22|  388|
|    Noida|  28.57|  77.32|  273|
|New Delhi|  28.65|  77.12|  213|
|New Delhi|  28.59|  77.17|  200|
|    Noida|    0.0|    0.0|  176|
|  Gurgaon|  28.49|  77.09|  158|
|    Noida|  28.57|  77.33|  155|
|New Delhi|  28.53|  77.22|  145|
|New Delhi|  28.64|  77.18|  140|
+---------+-------+-------+-----+
only showing top 10 rows
```

Getting Top Food hubs:

```
>>> top_hubs = food_hubs.filter(col("count") > 50)
>>> top_hubs.show()
+-------+-------+-----+
|lat_bin|lon_bin|count|
+-------+-------+-----+
|    0.0|    0.0|  969|
|  28.63|  77.22|  388|
|  28.57|  77.32|  273|
|  28.65|  77.12|  213|
|  28.59|  77.17|  200|
|  28.49|  77.09|  158|
|  28.57|  77.33|  155|
|  28.53|  77.22|  145|
|  28.64|  77.18|  140|
|   28.6|  77.23|  132|
```

7. Data Quality & Standardization Pipeline

Normalization of the given dataset:

```
>>> df = df.filter(col("Cuisines").isNotNull())
>>> df = df.withColumn("City", trim(col("City")))
>>> df = df.dropDuplicates(["Restaurant ID"])
>>> df = df.filter(~col("Currency").isin("Yes", "No"))
>>> df = df.withColumnRenamed("Average Cost for two", "avg_cost_for_two") \
...         .withColumnRenamed("Aggregate rating", "rating")
>>> df = df.filter(col("Aggregate rating").isNotNull())
>>> df_clean = df \
...     .filter(col("Cuisines").isNotNull()) \
...     .filter(col("Latitude").isNotNull() & col("Longitude").isNotNull()) \
...     .filter(~col("Currency").isin("Yes", "No")) \
...     .dropDuplicates(["Restaurant ID"]) \
...     .withColumn("City", trim(col("City")))
```

8. Locality-wise Price Range Distribution

Analysing range distribution using When:

```
>>> from pyspark.sql.functions import when, col
>>>
>>> df_price = df.withColumn(
...     "Price_Category",
...     when(col("Price range") == 1, "Cheap")
...     .when(col("Price range") == 2, "Affordable")
...     .when(col("Price range") == 3, "Expensive")
...     .otherwise("Luxury")
... )
>>> locality_counts = df_price.groupBy("Locality", "Price_Category") \
...                            .count()
>>> from pyspark.sql.functions import sum
```

```
>>> total_per_locality = locality_counts.groupBy("Locality") \
...                                     .agg(sum("count").alias("total"))
>>> df_final = locality_counts.join(total_per_locality, "Locality") \
...     .withColumn(
...         "percentage",
...         round((col("count") / col("total")) * 100, 2)
...     )
>>> df_final.select("Locality", "Price_Category", "percentage") \
...         .orderBy("Locality", col("percentage").desc()) \
...         .show(20)
```

Output Table:

Locality	Price_Category	percentage
ILD Trade Centre...	Affordable	50.0
ILD Trade Centre...	Cheap	50.0
12th Square Build...	Expensive	100.0
A Hotel, Gurdev N...	Affordable	100.0
ARSS Mall, Paschi...	Affordable	100.0
Aaya Nagar	Affordable	100.0
Abu Dhabi Mall, T...	Luxury	100.0
Abu Shagara	Expensive	66.67
Abu Shagara	Luxury	33.33
Acropolis Mall, K...	Expensive	100.0
Adajan Gam	Expensive	66.67
Adajan Gam	Affordable	33.33
Adchini	Expensive	38.46
Adchini	Cheap	38.46
Adchini	Affordable	23.08

Output of Price range distribution in Pivot table:

```
>>> df_pivot = df_final.groupBy("Locality") \
...     .pivot("Price_Category") \
...     .sum("percentage")
>>>
>>> df_pivot.show()
+--------------------+----------+-----+---------+------+
|            Locality|Affordable|Cheap|Expensive|Luxury|
+--------------------+----------+-----+---------+------+
|Piccadily Hotel, ...|      null| null|     null| 100.0|
|             Mt Eden|     33.33| null|    33.33| 33.33|
|               Tebet|      null| null|    100.0|  null|
|Premier Inn, Shal...|      null| null|     null| 100.0|
|           Tollcross|     100.0| null|     null|  null|
|           Sector 14|     34.62|61.54|     3.85|  null|
|             Adchini|     23.08|38.46|    38.46|  null|
|             Parnell|      null| null|    100.0|  null|
|          Rushikonda|     100.0| null|     null|  null|
| Plaza Mall, MG Road|      null| null|     null| 100.0|
```

9. Price Range Prediction (Classification)

```
>>> dt = dt.withColumn(
... "predicted_range",
... when(col("Average Cost for two") > 3000, 4)
... . when((col("Average Cost for two") > 1500) & (col("Aggregate rating")>4.0),
 3)
... .when (col("Average Cost for two") > 500 , 2)
... .otherwise(1)
... )
```

```
>>> dt.select("Average Cost for two", "Aggregate rating", "Price range", "predic
ted_range").show(20)
+--------------------+----------------+-----------+---------------+
|Average Cost for two|Aggregate rating|Price range|predicted_range|
+--------------------+----------------+-----------+---------------+
|                1100|             4.8|          3|              2|
|                1200|             4.5|          3|              2|
|                4000|             4.4|          4|              4|
|                1500|             4.9|          4|              2|
|                1500|             4.8|          4|              2|
|                1000|             4.4|          3|              2|
|                2000|               4|          4|              2|
|                2000|             4.2|          4|              3|
|                6000|             4.9|          4|              4|
|                1100|             4.8|          3|              2|
|                 800|             4.9|          3|              2|
|                 900|             4.2|          3|              2|
|                 800|             4.8|          3|              2|
```

```
>> correct = dt.filter(col("Price range") == col("predicted_range")).count()
>> total = dt.count()
>> accuracy = correct * 1.0 / total
>> print(accuracy)
.682151845638
>>
```

10. Top-N Recommendation Engine (Content-Based)

```
>>> from pyspark.sql.functions import col
>>> target_row = df.filter(col("Restaurant Name") == target_name).collect()[0]
>>> target_city = target_row["City"]
>>> target_cost = target_row["Average Cost for two"]
>>> target_rating = target_row["Aggregate rating"]
>>> target_votes = target_row["Votes"]
>>> target_name = "Pizza Hut"
>>> same_city_df = df.filter(df.City == target_city)
>>> from pyspark.sql.functions import udf
>>> from pyspark.sql.types import DoubleType
>>> import math
>>>
>>> def distance(cost, rating, votes):
...     return float(math.sqrt(
...         (cost - target_cost)**2 +
...         (rating - target_rating)**2 +
...         (votes - target_votes)**2
```

```
>>> recommendations = similar_df \
...     .filter(similar_df["Restaurant Name"] != target_name) \
...     .orderBy("distance") \
...     .limit(5)
>>> recommendations.show()
```

11. Optimized Currency Conversion (Broadcast Join)

```
>>> currency_df = spark.createDataFrame(currency_data, ["Currency", "rate_to_inr"])
>>>
>>> currency_df.show()
+--------+-----------+
|Currency|rate_to_inr|
+--------+-----------+
|     INR|        1.0|
|     USD|       83.0|
|     EUR|       90.0|
|     GBP|      105.0|
|     AED|       22.0|
|      QR|       23.0|
|     ZAR|        4.5|
|     BRL|       16.0|
|     TRY|        2.8|
+--------+-----------+

>>> from pyspark.sql.functions import broadcast
>>>
>>> joined_dt = dt.join(
...     broadcast(currency_df),
...     dt.currency_code == currency_df.Currency,
...     "inner"
... )
>>> dt.select("currency_code").distinct().show()
+-------------+
|currency_code|
+-------------+
|          IDR|
|          AED|
|           ??|
|          LKR|
```

File Edit View Search Terminal Help

```
>>> from pyspark.sql.functions import col
>>>
>>> final_df = joined_dt.withColumn(
...     "cost_in_inr",
...     col("Average Cost for two") * col("rate_to_inr")
... )
>>>
>>> final_df.show(10)
+--------------------+---------+--------------------+-------------+--------+-----------+-----------+
|     Restaurant Name| Currency|Average Cost for two|currency_code|Currency|rate_to_inr|cost_in_inr|
+--------------------+---------+--------------------+-------------+--------+-----------+-----------+
|Austin's BBQ and ...|Dollar($)|                  25|          USD|     USD|       83.0|     2075.0|
| BJ's Country Buffet|Dollar($)|                  10|          USD|     USD|       83.0|      830.0|
|       Cookie Shoppe|Dollar($)|                   0|          USD|     USD|       83.0|        0.0|
|El Vaquero Mexica...|Dollar($)|                   0|          USD|     USD|       83.0|        0.0|
|Elements Coffee C...|Dollar($)|                  10|          USD|     USD|       83.0|      830.0|
|Pearly's Famous C...|Dollar($)|                   0|          USD|     USD|       83.0|        0.0|
|         Chick-fil-A|Dollar($)|                  10|          USD|     USD|       83.0|      830.0|
|Guang Zhou Chines...|Dollar($)|                  10|          USD|     USD|       83.0|      830.0|
```